

A Size-Invariant Deep Reinforcement Learning Policy for the Interval Job Shop Scheduling Problem

Hernán Díaz [TODO: co-authors]

Draft — July 2026

Abstract

We present a constructive deep reinforcement learning (DRL) policy for the Interval Job Shop Scheduling Problem (IJSP), in which task processing times are only known to lie within closed intervals. The policy scores eligible operations through a Deep Sets encoder over dimensionless, per-operation uncertainty-aware features combined with a pooled global context, making both the policy and value networks *independent of the instance size*: a single trained network dispatches instances of any number of jobs and machines. Trained with PPO on only four 20×15 Taillard IJSP instances, the policy reaches a relative error of 13.4% (mean of three seeds; 12.3% best) with respect to the crisp Taillard lower bounds on unseen 20×15 instances, clearly outperforming the best priority dispatching rule (MOR, $\approx 46\%$) while producing solutions in milliseconds. Zero-shot, the same network beats MOR on every tested instance of all seven Taillard size classes, from 15×15 to 50×20 . Controlled ablations show that (i) sampling-based best-of- N inference provides a consistent improvement, (ii) self-attention over the eligible operations does not improve over pooling at practical training budgets, and (iii) a specialist trained on a single size class generalizes better across sizes than a multi-size-trained counterpart, even at an equalized per-instance budget. Finally, we characterize the quality of the policy as a population initializer for a state-of-the-art tabu search, laying out a pre-registered hybridization study.

1 Introduction

The Job Shop Scheduling Problem (JSP) is a classical NP-hard combinatorial optimization problem. In many practical settings, task durations are not known precisely at scheduling time; the *interval* JSP (IJSP) models this uncertainty by specifying each processing time as a closed interval, and asks for a robust operation ordering evaluated under interval arithmetic [2, 1]. State-of-the-art IJSP solvers are metaheuristics—artificial bee colony variants [1, 2] and, more recently, tabu search with specialized neighbourhoods [3]—which optimize each instance individually under a per-instance search budget of minutes.

An orthogonal line of research learns *constructive* policies for scheduling with deep reinforcement learning (DRL) [5, 6]. Such policies build a schedule in a single dispatching pass (milliseconds to seconds) and, once trained, amortize their cost across instances. To our knowledge, no learned constructive policy had been evaluated on the IJSP.

This paper makes the following contributions:

1. A **size-invariant DRL policy for the IJSP**: per-operation features are dimensionless (normalized by an instance lower bound) and uncertainty-aware; a Deep Sets [7] encoder with pooled global context yields policy and value networks whose parameters are independent of the numbers of jobs and machines.
2. An **empirical scaling and generalization study** on the Taillard IJSP benchmark: the policy improves monotonically with training budget on its training size class and transfers zero-shot to all seven size classes, dominating the best dispatching rule everywhere.
3. **Controlled ablations** with paired seeds and deterministic anchors: best-of- N inference, self-attention versus pooling, and specialist versus multi-size training.

4. A **pre-registered hybridization design** that uses the policy to seed the initial population of a state-of-the-art tabu search, including a characterization of seed-pool quality against randomized dispatching baselines.

2 Related Work

Interval JSP. The IJSP evaluates schedules with interval arithmetic and ranks interval makespans either by their expected value (midpoint) or lexicographically by upper bound [2]. Population-based metaheuristics constitute the state of the art: *fEABC* [1] and *ESABC* [2] report mean relative errors of roughly 5–14% on the Taillard IJSP set, and irace-tuned tabu search reduces this to 0–12% depending on the size class [3].

DRL for scheduling. Learning-to-dispatch approaches encode the scheduling state and score eligible operations with neural networks trained by policy-gradient methods [5]; attention-based encoders are standard in neural combinatorial optimization [6]. Sampling multiple rollouts at inference (best-of- N) is a common, effective decoder [6]. Our work differs in the uncertainty setting (intervals), in the explicitly dimensionless feature design that yields size invariance for both heads, and in the systematic ablation methodology.

3 The Interval Job Shop Scheduling Problem

An IJSP instance consists of n jobs and m machines; each job is a fixed sequence of m operations, one per machine, and each operation o has an interval duration $\mathbf{p}_o = [p_o^L, p_o^U]$. A solution is an operation processing order compatible with job precedences; its *semiactive* schedule assigns to each operation the earliest start compatible with its job and machine predecessors, computed with interval arithmetic: $\mathbf{s}_o = \max(\mathbf{c}_{jp(o)}, \mathbf{c}_{mp(o)})$ with the component-wise maximum, and $\mathbf{c}_o = \mathbf{s}_o + \mathbf{p}_o$. The interval makespan $\mathbf{C}_{\max}$ is the lexicographic maximum (by upper bound, then lower) of job completions. Following [2], solutions are ranked during optimization by the worst case $C_{\max}^U$ (with the lower bound as tie-breaker), and reported using the expected makespan $E[\mathbf{C}_{\max}]$ (interval midpoint) through the relative error

$$\text{RE}(\%) = \frac{E[\mathbf{C}_{\max}] - \text{LB}}{\text{LB}} \times 100,$$

where LB is the published lower bound of the crisp Taillard counterpart [4]. As in prior IJSP work, these crisp bounds are an indicative, hardware-independent yardstick rather than a strict optimality gap.

4 Method

4.1 Dispatching as a Markov decision process

A schedule is built in nm steps. At each step the agent observes the set of *eligible* operations (the next unscheduled operation of each unfinished job) and selects one; the environment appends it to the schedule and updates job and machine completion intervals. The reward combines a terminal makespan term with dense shaping components (idle time, load balance, local bound improvement) whose weights were tuned by a benchmark-gated search; all reward components collapse intervals by their worst case.

4.2 Size-invariant state representation

Each eligible operation is described by a 16-dimensional vector of *dimensionless* quantities: duration, earliest start and remaining job work (lower and upper bounds divided by the instance lower bound LB), their relative interval widths, job progress, remaining operation count fraction, machine context

(remaining machine load /LB, potential idle gap /LB, relative machine congestion), slack against the earliest eligible operation, and normalized machine/job completion times. A 12-dimensional global context aggregates the schedule state exclusively through means, maxima and standard deviations of per-machine and per-job quantities, plus the pending relative uncertainty. No feature depends on n or m through its dimension.

4.3 Architecture

A shared MLP ϕ (two layers, width 128, LayerNorm) embeds each eligible operation. The global embedding g is an MLP over the concatenation of masked mean- and max-pooling of $\{\phi_i\}$ with the global features (a Deep Sets construction [7]). The policy head scores each candidate as $\text{MLP}([\phi_i; g])$ followed by a masked softmax; the value head is an MLP over g alone. Both heads are therefore size-invariant. An optional stack of pre-LN self-attention blocks over $\{\phi_i\}$ (evaluated in Section 6.3) can be inserted before pooling.

4.4 Training and inference

The agent is trained with PPO [8] using minibatch updates (256 transitions, 4 epochs), GAE ($\lambda=0.95$), advantage normalization, gradient clipping, orthogonal initialization and $\gamma=1$ (finite horizon, terminal-dominated reward). Training alternates sampled episodes with periodic greedy evaluations used for model selection. At test time we use **best-of- N** inference: one greedy rollout plus $N-1$ policy samples, returning the best solution by worst case ($N=64$ throughout).

5 Experimental Setup

Instances and data split. We use the Taillard IJSP benchmark (TA1–TA70, seven size classes) [2, 4]. The final model is trained on **TA11–TA14** only (20×15); **TA15–TA20** served as the development set for all configuration decisions and are reported separately; all remaining instances are entirely unseen. Three independent training seeds are reported as mean and best.

Protocol. All comparisons run under a fixed benchmark harness with per-seed bit-reproducibility. Deterministic dispatching rules (SPT, LPT, MOR, MWKR) act as *anchors*: their results are verified to be identical across all code versions, guaranteeing that improvements are attributable to learning and not to environment changes. Configuration changes were accepted only under a pre-registered two-stage rule (a cheap filter benchmark, then a full confirmation benchmark).

Baselines. Priority dispatching rules (best: MOR); published IJSP solvers fEABC [1], ESABC [2] and tabu search [3] as reference yardsticks (30 runs, minutes of search per instance, different computational class).

6 Results

6.1 In-size performance and scaling

Figure 1 and Table 1 show the development of mean RE on TA15–TA20 as the per-instance training budget grows. The policy scales monotonically (28.0% at 100 episodes, 16.4% at 300, 13.4% at 1000) with clearly diminishing returns at the last step, locating the pure-budget ceiling around 11–12%. Seed variance collapses with budget (e.g. per-instance standard deviations of 13–24 makespan units at 300 episodes). Best-of-64 inference contributes a consistent improvement over a single greedy rollout (−3.6% mean makespan, better on 3/6 development instances with none worse, at its introduction point).

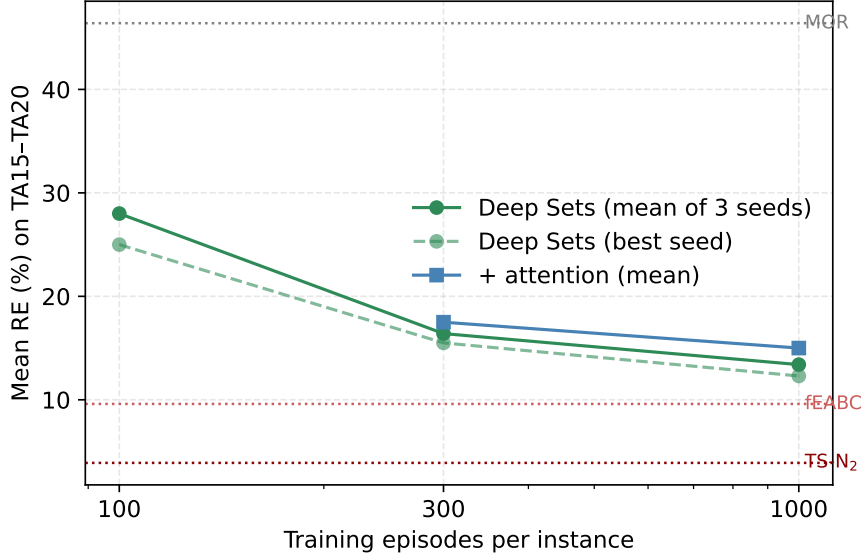


Figure 1: Mean RE on the development set (TA15–TA20) versus training episodes per instance. Dotted lines: best dispatching rule (MOR) and published metaheuristics (30 runs, minutes per instance) as reference yardsticks.

Table 1: Mean [best-seed] RE (%) on TA15–TA20 (development set).

Configuration	100 eps	300 eps	1000 eps
v2 Deep Sets	28.0 [25.0]	16.4 [15.5]	13.4 [12.3]
v2 + attention (2 blocks)	—	17.5 [16.1]	15.0 [13.9]
MOR (best rule)	46.4		
fEABC [1] / TS-N ₂ [3]	9.6 / 3.9 (search-based reference)		

6.2 Zero-shot cross-size generalization

Because no network dimension depends on the instance size, the 20×15 specialist evaluates directly on any size class. Figure 2 reports RE on unseen instances of all seven classes: the policy beats MOR on **every** tested instance, with RE ranging from 7.0% (15×15) to 24.5–25.0% on the hardest transfer classes (30×20 , 50×15), and no degradation cliff up to 1000-operation episodes (50×20 : 15.1% versus MOR’s 48.7%).

6.3 Ablations

Attention versus pooling. Adding two self-attention blocks over the eligible operations—identical code path, one flag—yields parity at 300 episodes (+1.2%, all six development instances within noise) and parity-to-worse at 1000 (+1.6%, one instance significantly worse), at $\sim 30\%$ higher per-episode cost (Table 1). At these budgets, pooled context already captures the necessary interaction information.

Specialist versus multi-size training. Training one agent on twelve instances of three size classes (15×15 , 20×15 , 20×20) loses to the single-size specialist on *all* five common evaluation instances—including the size classes the multi-size agent itself trained on—both at matched total budget (3000 episodes) and at matched *per-instance* budget (12,000 episodes; e.g. 9.5 vs. 8.7 on TA5, 18.5 vs. 15.9 on TA25-class, 28.3 vs. 24.5 on TA41-class). Per-instance depth dominates size diversity, and specialist generalization is the stronger phenomenon.

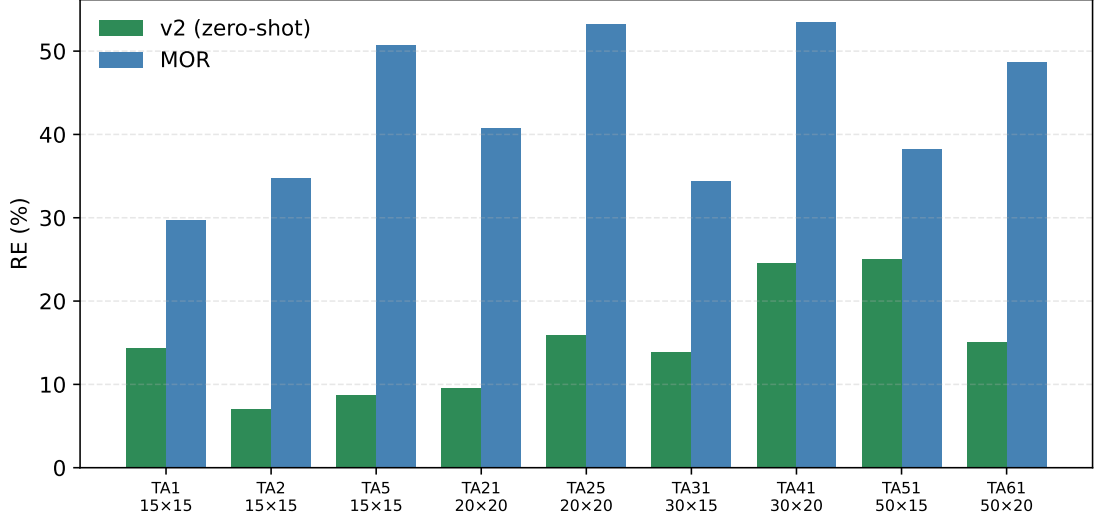


Figure 2: Zero-shot RE by instance for the 20×15 -trained specialist (best-of-64) versus the MOR dispatching rule.

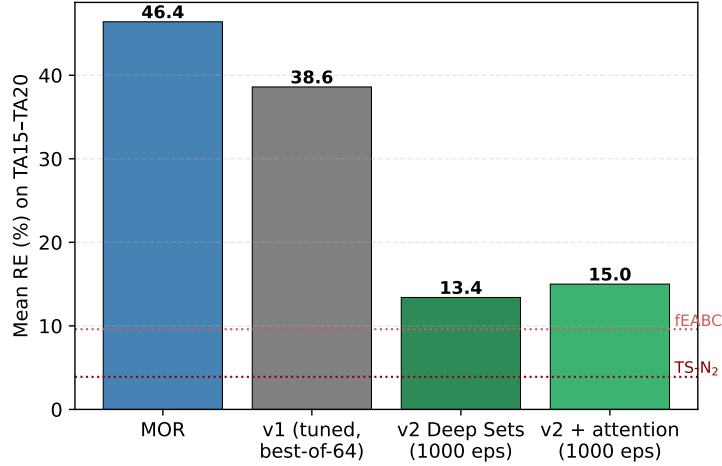


Figure 3: In-size ablation summary (mean RE, TA15–TA20, 1000 episodes). The tuned v1 flat-feature agent is shown for reference.

6.4 Positioning against search-based solvers

The learned policy occupies a distinct computational class: it constructs a solution in milliseconds (best-of-64: seconds) versus minutes of per-instance search for the metaheuristics. Its final in-size RE (13.4%) remains above fEABC (9.6%) and far from tuned tabu search (3.9%); its value lies in amortized speed, in full cross-size generality from a single network, and—as explored next—as an initializer for search.

7 Toward Hybridization: Seeding Tabu Search Populations

The tabu search of [3] starts each run from a population of 250 individuals. We propose seeding a fraction p of this population with policy samples. Figure 4 characterizes seed-pool quality on three clean pilot instances (excluded from RL training/development and from the TS irace tuning): the *median* policy sample is better than the *best* individual produced by randomized dispatching baselines (MOR with ϵ -greedy noise, and a mixed-rule GRASP-style generator) by 15–25 RE points.

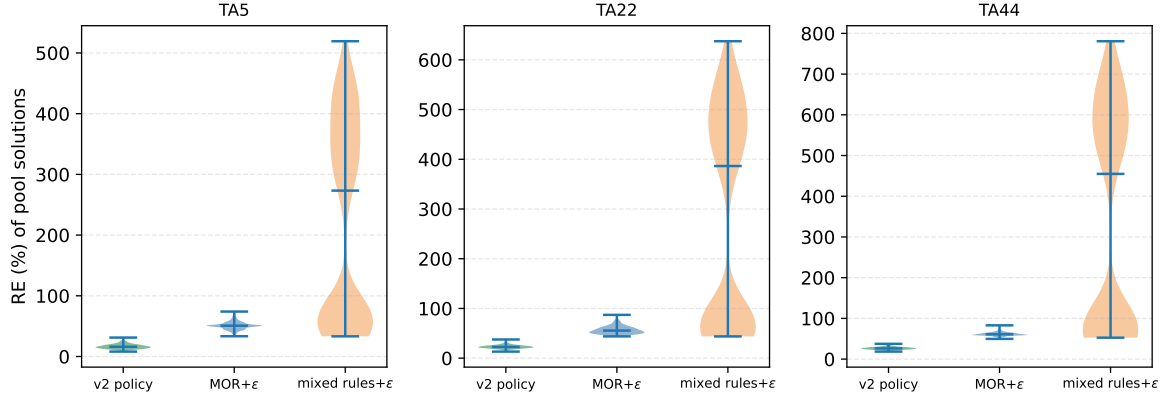


Figure 4: Seed-pool quality (RE of 1024 sampled solutions per generator) on the three pilot instances.

The pre-registered pilot compares five arms (control; $p \in \{10, 50, 100\%\}$ with policy seeds; $p=50\%$ with MOR- ϵ seeds) under paired RNG seeds, recording anytime curves. Hypotheses: (H1) policy seeding reduces time-to-target; (H2) the benefit exceeds that of heuristic seeding at equal p ; (H3) an intermediate p is optimal due to diversity loss at $p=100\%$. [TODO: pilot results pending]

8 Conclusions

We introduced a size-invariant constructive DRL policy for the IJSP that dominates priority dispatching rules across the entire Taillard grid from a single network trained on four instances of one size class, and characterized its scaling, its inference-time sampling gains, and two negative results of methodological value (attention parity; specialist over multi-size training). The policy’s sample distributions position it as a strong population initializer for state-of-the-art search, which the ongoing hybridization study quantifies. Future work includes training-time curricula exploiting the size invariance, uncertainty-heterogeneous instances where ranking criteria diverge, and end-to-end integration with neighbourhood search.

References

- [1] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. Fast elitist ABC for makespan optimisation in interval JSP. *Natural Computing*, 22(4):645–657, 2023.
- [2] H. Díaz, J. J. Palacios, I. González-Rodríguez, C. R. Vela. An elitist seasonal artificial bee colony algorithm for the interval job shop. *Integrated Computer-Aided Engineering*, 30(3):223–242, 2023.
- [3] H. Díaz et al. Neighbourhood structures and ranking operators for the interval job shop scheduling problem. Under review, 2026.
- [4] E. Taillard. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 64(2):278–285, 1993.
- [5] C. Zhang, W. Song, Z. Cao, J. Zhang, P. S. Tan, C. Xu. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.
- [6] W. Kool, H. van Hoof, M. Welling. Attention, learn to solve routing problems! *ICLR*, 2019.
- [7] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Póczos, R. Salakhutdinov, A. Smola. Deep sets. *NeurIPS*, 2017.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms. *arXiv:1707.06347*, 2017.